

iptables 사용자 지정 체인 완벽 해부 가이드

복잡한 방화벽 규칙을 함수처럼 관리하는 방법



| 0. 서론: 왜 굳이 '체인'을 나누는가?

혼돈의 INPUT 체인

모든 규칙을 INPUT 체인 하나에 다 넣으면 발생하는 문제점:

- ⚠ 규칙이 100개, 1,000개로 무한정 늘어납니다.
- ⚠ `iptables -L` 입력 시 스크롤 압박으로 멘탈이 터집니다.
- ⚠ 어떤 규칙이 어디 있는지 찾기 힘들어 유지보수가 불가능해집니다.

| 해결책: 사용자 지정 체인 (Custom Chain)

정리정돈의 마법

특정 목적(예: 핑 차단, 스캔 감지)을 가진 규칙들을 별도의 '방'으로 격리합니다.

사용자 지정 체인의 3대 장점

1. **가독성 향상:** 코드가 깔끔해져서 읽기 쉽습니다.
2. **수정 용이:** 해당 체인(함수)만 건드리면 됩니다.
3. **성능 이득:** 필요 없는 패킷은 검사하지 않고 넘길 수 있습니다.

| 개념 잡기: 프로그래밍에 비유하면?



INPUT 체인

프로그램의 시작점인
main() 함수

```
void main() { ... }
```



사용자 지정 체인

특정 기능을 수행하는
사용자 정의 함수(Function)

```
void block_ping() { ... }
```


| iptables 명령어 해부학

본격적인 실습에 앞서, 우리가 사용할 명령어의 기본 뼈대를 이해해야 합니다.

```
iptables -A CHAIN_NAME -p protocol -j TARGET
```

- **iptables** : 리눅스 커널 방화벽을 설정하는 도구입니다.
- **-A** : Append. 규칙을 체인의 **맨 끝**에 추가합니다.
- **CHAIN_NAME** : 규칙이 들어갈 방(체인)의 이름입니다.
- **-p** : Protocol. 어떤 패킷을 잡을지 조건을 겁니다.
- **-j** : Jump. 조건이 맞으면 어떤 행동(Target)을 할지 정합니다.



실습 1. 핑(ICMP) 참교육하기

특정 대역(1.1.1.x)에서 오는 핑 공격을 감지하고 차단하는 시나리오

| Step 1: 체인 생성 (그릇 만들기)

먼저 규칙을 담을 새로운 그릇(체인)을 만들어야 합니다.

```
iptables -N TEST
```

-N New Chain

새로운 사용자 정의 체인을 생성합니다. 이름은 'TEST'입니다.

※ 현재 상태: 이름만 있는 빈 깡통입니다. 아직 아무런 기능도 없습니다.

| Step 2: 내부 규칙 정의 (함수 코딩)

이제 TEST 체인 안에 실제 동작할 규칙을 넣습니다.

첫 번째 규칙은 "로그 남기기"입니다. 바로 차단하지 않고, CCTV처럼 기록부터 합니다.

```
iptables -A TEST \  
-s 1.1.1.0/24 \  
-p icmp --icmp-type echo-request \  
-j LOG --log-prefix "[ICMP_BLOCK] "
```

이 긴 명령어를 하나씩 해부해 봅시다.

해부 1: 위치와 대상 지정

```
-A TEST -s 1.1.1.0/24
```

📍 -A TEST

Append to TEST

TEST 체인의 목록 맨 끝에 이 규칙을 붙이라는 뜻입니다. 순서가 중요하기 때문에 주로 -A를 씁니다.

🎯 -s 1.1.1.0/24

Source (출발지)

패킷의 출발지 IP가 1.1.1.0 ~ 1.1.1.255 대역인 경우에만 이 규칙을 적용합니다. 다른 곳에서 온 핑은 무시합니다.

| 해부 2: 프로토콜의 디테일

```
-p icmp --icmp-type echo-request
```

-p icmp : 감시할 프로토콜은 ICMP(인터넷 제어 메시지 프로토콜)입니다.

--icmp-type echo-request : 여기가 핵심!

핑(Ping)은 "야 있냐?(Request)"와 "어 있다(Reply)"로 나뉩니다.

- 우리는 **질문(Request)**만 막으면 됩니다.
- 응답(Reply)까지 막으면 내가 핑을 보냈을 때 돌아오는 것도 막힐 수 있습니다.

해부 3: 행동 개시 (Action)

```
-j LOG --log-prefix "[ICMP_BLOCK] "
```

■ -j LOG

Just Log

패킷을 차단하거나 허용하지 않습니다. 단지 시스템 로그 파일(/var/log/messages 등)에 기록만 하고, 패킷을 **다음 규칙으로 통과**시킵니다.

🔑 --log-prefix

꼬리표 붙이기

로그 파일은 텍스트의 바다입니다. 나중에 `grep "[ICMP_BLOCK]"` 명령어로 쉽게 찾기 위해 식별자를 붙여주는 센스입니다.

| 규칙 2: 차단 (뚝배기 깨기)

로그를 남겼으니 이제 진짜로 차단할 차례입니다.

```
iptables -A TEST ...조건 동일... -j DROP
```

DROP



"무시하기"

패킷을 조용히 버립니다.
상대방은 응답을 못 받아 타임아웃까지 기다려야 합니다.
(보안성 높음)

REJECT



"거절 통보"

"너 차단됨"이라는 에러 메시지를 즉시 보냅니다.
상대방이 차단 사실을 알게 됩니다.

| 규칙 3: 복귀 (억울한 놈들 구제)

```
iptables -A TEST -j RETURN
```

왜 필요한가?

TEST 체인에 들어왔는데, 위에서 설정한 조건(1.1.1.0 대역 등)에 해당하지 않는 패킷들은 어떻게 될까요?

이 규칙이 없으면 미아다 됩니다. RETURN을 만나면 다시 **자기를 호출했던 INPUT 체인**으로 돌아갑니다.

이것이 명시적인 흐름 제어입니다.

| Step 3: 연결 (함수 호출)

함수(TEST 체인)를 다 찢으면, main 함수(INPUT 체인)에서 호출을 해야 작동하겠죠?

```
iptables -A INPUT -j TEST
```

의미: "INPUT 체인으로 들어오는 모든 패킷을 일단 TEST 체인으로 보내봐라."

⚠ 주의: 지금은 조건 없이 다 보내지만, 실무에선 비효율적일 수 있습니다. (실습 2에서 개선)

| 실습 1 전체 흐름도





실습 2. 스텔스 스캔 탐지

해커들의 Nmap SYN Scan을 막아보자

| SYN 스캔(Stealth Scan)이란?

"똑똑" 하고 도망가기

정상적인 연결(3-Way Handshake)을 맺지 않는 비정상적인 스캔 방식입니다.

1. **SYN (요청)** 보냄 : "문 열렸니?"
2. 서버가 **SYN+ACK (응답)** 보냄 : "어, 열렸어."
3. 해커는 **RST (취소)** 보내고 연결 끊음 : "응 안 들어가~"

로그가 잘 안 남아서 '스텔스'라고 부릅니다. 일반적인 로그 설정으로는 잡기 힘들기 때문에 특별한 규칙이 필요합니다.

| 전용 체인 생성

스텔스 스캔만 전담하는 격리 병동을 만듭니다.

```
iptables -N STEALTH_SCAN
```

이름만 봐도 용도를 알 수 있게 명확하게 짓는 것이 좋습니다. 이제 이 안에 강력한 감시 규칙을 세팅할 것입니다.

| 로그 규칙: 이번엔 좀 복잡하다?

```
iptables -A STEALTH_SCAN \  
-p tcp --syn \  
-m state --state NEW \  
-m limit --limit 5/s --limit-burst 10 \  
-j LOG --log-prefix "[STEALTH_SYN] "
```

명령어가 길어졌습니다. 하지만 겁먹을 필요 없습니다. 하나씩 뜯어보면 논리적입니다. 특히 limit 모듈이 핵심입니다.

| 해부 1: 범인 식별 (TCP & State)

🔍 -p tcp --syn

SYN 플래그 감지

TCP 패킷 중에서 연결 요청을 의미하는 SYN 비트가 1인 패킷만 잡습니다. 이것이 바로 스캔의 시작점이기 때문입니다.

🕒 -m state --state NEW

새로운 연결만

이미 연결되어 통신 중인 패킷은 건드리지 않고, **새롭게 들어오는(NEW)** 시도만 감시합니다. 오탐을 획기적으로 줄여줍니다.

| 해부 2: limit 모듈 (핵심 개념)

로그 폭주를 막아라!

만약 해커가 작성하고 초당 1만 번 스캔하면 어떻게 될까요?

시나리오:

1. 로그도 초당 1만 줄이 쌓입니다.
2. 디스크가 순식간에 꽉 찹니다 (Disk Full).
3. 서버가 다운됩니다 (DoS 상태).

방화벽이 오히려 서버를 죽이는 무기가 될 수 있습니다. 이를 막기 위해 "**수도꼭지 조절**"을 하는 것이 limit 모듈입니다.

| limit 파라미터 상세 분석

```
-m limit --limit 5/second --limit-burst 10
```



--limit-burst 10

"물통의 크기"

처음에는 패킷 **10개까지는 제한 없이** 즉시 통과(로그 기록)시킵니다. (관용 구간)



--limit 5/second

"리필 속도"

물통의 빈 자리가 **1초에 5개씩** 채워집니다. 이 속도보다 빨리 들어오면 로그를 찍지 않고 그냥 버립니다.

| 로그 규칙 완성

논리 흐름:

1. TCP이고 SYN 패킷인가? (YES)
2. 새로운 연결(NEW)인가? (YES)
3. 최근에 너무 많이 찍히진 않았나? (Limit 체크)
4. 위 조건을 모두 통과했다면 → **LOG 기록**

이렇게 하면 아무리 공격이 쏟아져도 로그 파일은 일정한 속도로만 기록되어 시스템을 보호합니다.

| 규칙 2: 차단 (DROP) - Limit을 뺀다?

```
iptables -A STEALTH_SCAN ...조건 동일... -j DROP
```

❗ 잠깐! 여기서 Limit을 뺍니다.

만약 차단 규칙에 limit를 걸면 어떻게 될까요?

"초당 5개까지만 차단하고, 그 이상은 통과시킨다"는 끔찍한 결과가 나옵니다. (Limit은 매칭이 안 되면 다음 규칙으로 넘기니까요)

스캔은 무조건 막아야 하므로 DROP 규칙에는 limit을 절대 쓰지 않습니다.

| Step 3: 스마트한 연결 (Optimization)

실습 1처럼 무조건 보내지 않고, 조건에 맞는 놈만 보냅니다.

```
iptables -A INPUT \  
-p tcp --syn -m state --state NEW \  
-j STEALTH_SCAN
```

필터링 후 점프:

이미 연결된 패킷(Established)이나 UDP 패킷 등은 굳이 사용자 체인에 보낼 필요가 없습니다. 여기서 한 번 걸러주면 불필요한 체인 이동을 줄여 **CPU 낭비를 막을 수 있습니다**. 이것이 프로의 방식입니다.

| 왜 최적화가 중요한가?

패킷은 초당 수만 개

방화벽 규칙은 들어오는 **모든 패킷마다** 하나씩 검사합니다.

만약 규칙이 1,000개라면 패킷 하나당 1,000번 비교해야 합니다. 엄청난 부하가 걸리겠죠?

사용자 지정 체인과 조건부 점프(Jump)를 잘 쓰면, 비교 횟수를 획기적으로 줄여 **네트워크 지연(Latency)**을 방지합니다.



High Performance



Low CPU Usage



뒷정리 (설거지)

만드는 것만큼 중요한 지우는 순서

| 1. 내용 비우기 & 2. 연결 끊기

🏠 1. Flush (-F)

```
iptables -F TEST
```

체인 안의 모든 규칙을 삭제합니다. 마치 폴더 안의 파일을 모두 지우는 것과 같습니다.

✂ 2. Delete Rule (-D)

```
iptables -D INPUT -j TEST
```

INPUT 체인에서 해당 체인을 호출하던 연결 고리를 끊습니다. 이것 안 하면 체인을 못 지웁니다.

| 3. 체인 삭제 (-X) 및 주의사항

```
iptables -X TEST
```

이제 빈 깡통이 된 체인을 완전히 삭제합니다.

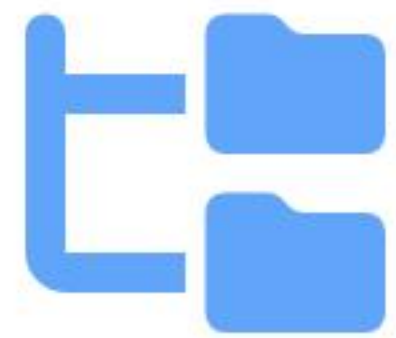
✕ 자주 보는 에러: "Device or resource busy"

이 에러는 100% 다음 두 가지 중 하나입니다.

1. 체인 안에 규칙이 남아있거나 (Flush 안 함)
2. 누군가 이 체인을 참조하고 있거나 (Link 안 끊음)

반드시 위에서 설명한 순서(Flush -> Delete Rule -> Delete Chain)를 지켜야 합니다!

| 최종 요약



함수화

사용자 지정 체인은
정리 정돈된 폴더이자
함수입니다.



순서 중요

LOG 먼저, **DROP** 나중.
죽은 자는 말이 없습니다.



Limit 활용

로그는 **Limit** 필수,
차단은 **Limit** 금지.
목적을 명확히 하세요.

감사합니다!